Project 2: Mean-Shift Tracker

Katarina Brešar

I. Introduction

In this assignment we implement the mean shift algorithm for seeking the modes of a distribution. We then apply this algorithm to the problem of tracking an object in a video sequence. We test it on different sequences and analyze the results.

II. Experiments

A. Mean-shift mode seeking

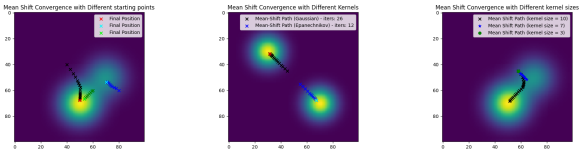
First, we implemented the mean shift algorithm and tested it on two synthetic response maps. We first tested how the starting point affect the convergence and the final position of the algorithm. In figure 1a we can see how the choice in starting position affects the convergence. All three starting positions convergence towards a mode, but when the starting point is closer to the local maximum it will converge towards it instead of the proper mode, since mean-shift is a greedy algorithm that cannot see the global structure. In figure 1b we can see how the choice of kernel affects the convergence. To test this we created a new function that has two distinct peaks (both with same value). The starting points are at the same distance from the two peaks. The mean shift with the gaussian kernel converges towards the top left peak and while it takes a smoother path it also takes more iterations to converge, while on the other hand using the epanechnikov kernel it covnerges towards the bottom right peak and it takes a more direct path and less iterations to converge. This happens since the epanechnikov kernel has a constant derivative so a more steady step size, whereas the gaussian kernel's derivative is again a gaussian which leads to smaller (smoother) steps as it gets closer to the peak. In figure 1c it's clear that if the size of the kernel is too small it will not converge towards the mode (not even the local maximum). In this case setting the kernel size to 7 was still too small and started to converge towards the local maximum, but kernel of size 10 was large enough to overlap with the gradient of the global mode and therefore it converged towards it.

To speed up convergence we can add a relaxing factor to the mean shift update. So instead of just updating the new position of the mean we add a factor $\lambda > 1$ so that: $x_{k+1} = x_k + \lambda m(x_k)$. But this can lead to overshooting if the factor is too high. To fix that we compare the relaxed position to the non-relaxed position and if it's larger (so closer to the mode) we use that, if not we use the non-relaxed position and reduce the relaxation by 20%. Note that using $\lambda = 1$ is the same as not using relaxation at all. In table I we see that the higher the factor is the faster the convergence, but if it's too high (despite the corrections) it can lead to overshooting and therefore a worse final position.

Relax Factor	Iterations	Distance from True Mode
1.0	21	4.58
1.5	15	5.84
3.0	10	3.34
5.0	9	0.93
6.5	9	1.79
8.0	6	0.83

Table I: Speed of convergence and distance from true mode with different relax factors.

To show importance of the termination criteria we tested the mean shift with different values for the tolerance on a new response map. Instead of having a circular peak (with one mode) we created a response map with a more complex shape that has an area with a distinct maximum. Mean shift, as expected, converged towards the part that was closest to the starting point. But when the tolerance was high ($[1, 2]$) it converged in the right way but it stopped further from the true peak, whereas with a lower tolerance ($[0.01, 0.1]$) it took more steps but it got much closer to the true peak. In figure 2 we can see when the algorithm stopped for different values of the tolerance when all the other parameters stayed the same.



(a) Starting points. (b) Kernels. (c) Kernel sizes.

Figure 1: Mean shift convergence with change in parameters.

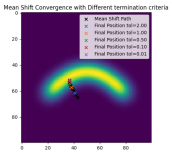


Figure 2: Mean shift convergence with different termination criteria.

B. Mean-shift tracker

The mean-shift tracker is a simple tracking algorithm that uses the mean shift algorithm to find the mode of the distribution of the target in the current frame. We first tested the tracker on five sequences from the VOT14 dataset. The tracking performance is shown in table II. In it we can see that the tracker performed well on all sequences losing the target only twice in the fernando and bolt sequences and once in the drunk sequence.

Sequence	Number of Failures	Frames per Second
drunk	1	587.5
fernando	2	189.3
bolt	2	450.5
car	0	562.4
polarbear	0	927.4

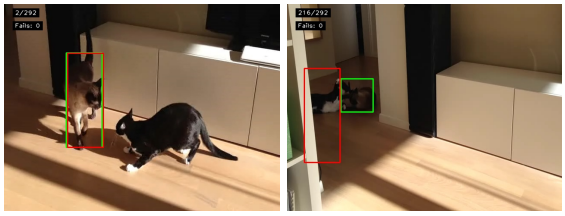
Table II: Tracking performance on five sequences.

1) Failure cases: The first kind of failures that we noticed in the tested sequences were cause by fast motion and dynamics of the target. Because MS is a local search algorithm if the object's center moves further than the kernel radius between two frames, the tracker will fail to find it (finds no or wrong gradient to follow). In the bolt sequence when the runner was moving fast and getting closer to the camera the tracker lost it. Interestingly in the car sequence the car was moving fast as well and getting closer to the camera but there were no failures. But since the car was getting bigger and bigger and the tracker stayed the same size, it was actually tracking just a part of the

car (in this case the back window) and not the whole car, so there were no failures but the overlap with the ground truth was not very good.

The second kind of failures were cause by appearance shift of the target or its background. Again looking at the bolt sequence, next to the fast motion change when the failure happened another runner entered the frame and the tracker likely got confused and started following the wrong target (as can be seen in figure 4a). In the fernando sequence the target was a cat fighting with another cat. The tracker failed when the second cat occluded the target one (as can be seen in figure 3b). So again an appearance shift happened and the tracker wrongly started following the second cat. Another example was in the drunk sequence where the target was a car driven by a drunk driver. When the car moved mostly out of frame and only a small part of it was visible the tracker failed to find it after it reappeared in full. So the histogram (that was tracking the back of the car before it went out of frame) changed a lot when it moved back in and the tracker failed.

The tracker performed well on the polarbear sequence, which makes sense since the target was moving slowly and the background stayed mostly the same. There was nothing to distrupt the tracker and the target was always visible (no change of appearance or dynamics).

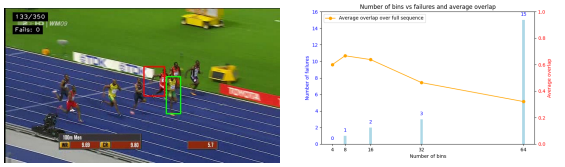


(a) Overlap 0.918 with GT. (b) Failure.

Figure 3: Failure example for fernando sequence.

To improve the robustness of the tracker we could add a scale estimation to it so that it can adapt to changes in the size of the target. To make sure it doesn't get confused by other objects that get into the background of the frame we could search for the features of the target by focusing on what discriminates it from the background instead of just looking at the histogram of the target.

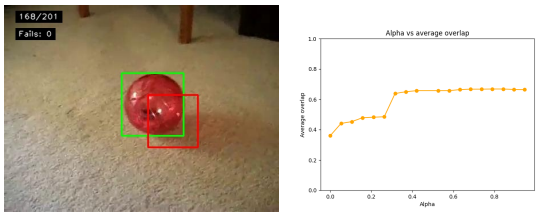
2) Setting the parameters: The mean shift tracker has several parameters that can affect its performance. The first one is number of bins in the histogram. In figure 4b we can see that with a high number of bins the model became too specific and when the background changed the histogram changed completely and the tracker failed. While the tracker didn't fail when the number of bins was small, the average overlap with the ground truth was lower since the model was too general.



(a) Bolt example. (b) Number of bins.

Figure 4: Tracking performance with changing parameters.

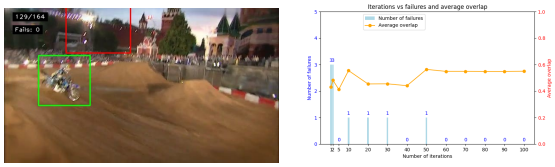
We also analyzed how the parameter alpha (that controls the update of the histogram model) affects the performance. We did that on the sphere sequence (shown in figure 5a) since it had no failures and we could see the effect of the parameter on the average overlap with the ground truth. As seen in figure 5b a low value of alpha (especially 0) lead to a lower average overlap since the model was not adapted and didn't adapt to change. And since in the video the target moves to a shadowy place and the camera gets closer, the histogram changes as well and without updating the histogram (or doing it so by a small alpha) the tracker overlaps poorly with the ground truth. But we can see that at around $\alpha = 0.5$ the overlap plateus so increasing alpha further doesn't lead to a better performance.



(a) Sphere example. (b) Alpha values.

Figure 5: Tracking performance with changing parameters.

Another parameter we were interested in is the number of iterations. This was tested on the motocross sequence example (shown in figure 6a) as it showed the biggest change in performance. As seen in figure 6b the overlap was mostly the same despite some having only 1 iteration, and some 100. It's important to note that with every failure the tracker reset the frame to the ground truth. But the number of failures here show why more iterations is better for accuracy. The failures stopped at 60 iterations. And since with increasing the number of iterations the speed of the tracker decreases, there is no need to do more than that.



(a) Motocross example. (b) Number of iterations

Figure 6: Tracking performance with changing parameters.

Another parameter that can affect the performance is the kernel size. If it's too small it can lead to failure since the tracker won't be able to follow the target from one frame to the next, but if it's too big it can lead to a worse performance since the tracker will be less precise. We had to adjust the width and height of the kernel for each of the mentioned sequences to get the best performance.

III. Conclusion

Mean shift is a simple and efficient algorithm for mode seeking and tracking. It has many parameters that can be adjusted to fit the specific problem and improve the performance. However it is a local search algorithm and can fail when the target moves too fast or there are changes in the appearance of the target or the background. There are specific situations in which the tracker fails and some where it performs well. To improve the performance we could add scale estimation or modeling the background aswell as the target.